Capstone Project: Amortized Bayesian Inference using Conditional Normalizing Flows

Vigneshwar Karthikeyan

Department of Physics, University of Cincinnati

PHYS 4099 : Physics Department

April 24, 2026

1 Motivation

A new sterile neutrino analysis as a part of the NuMI Off-axis ν_e Appearance (NOvA) experiment employs a global negative log-likelihood based fitter that is trying to fit for multiple oscillation parameters including mass splitting and mixing (Δm^2_{41}, $sin^2(\theta_{24})$, $sin^2(\theta_{34})$, δ_{24}, $sin^2(\theta_{23})$, Δm^2_{32}). Fitting for these parameters is tantamount to finding maxima on the likelihood surface. The problem, however, lies within our model of the systematic uncertainties. A naive fitter (like a simple Minuit Instance) is unable to differentiate between the added systematic uncertainties and actual sterile neutrino artifacts. As a result, the 3-flavor hypothesis is not the lowest χ^2 value for a given experiment. It is instead some combination of sterile neutrino parameters.

To find this best fit point, we currently employ a custom fitter that uses the *Hessian Matrix* to find minima in the χ^2 surface, but due to the sheer size and degenerate nature of the phase space (i.e, a large multi-modal χ^2 distribution for a given experiment), finding the global minimum is difficult. The current, brute-force approach the fitter uses is to seed the fit at randomly thrown points within uniform, and 3-flavor Gaussian constrained priors. The hope is that with a wide enough grid coverage, we can find the minima. As a result, the fitter sometimes does not converge at all, and at other times takes too long to do so.

A solution to this problem, as with daily use cases, is to suggest better starting points so that the fitter converges on a solution sooner. This capstone strives to provide a proof of principle for a better approach to this problem. We want to find a way to provide optimal starting points to the fitter so we decrease the required phase-space coverage.

2 Background

This analysis is frequentist in nature. We generate data, and we find the parameters that best describe the data through a fitting procedure. The primary idea behind this capstone is that we could enhance this frequentist analysis with a Bayesian framework. A traditional Bayesian analysis involves constructing a likelihood surface $P(\vec{d}|\vec{\theta})$ and inverting it using Bayes' theorem to find the posterior probabilities $P(\vec{\theta}|\vec{d})$. Finding the maxima in this posterior will tell us which data point(s) $\vec{d_0}$ are most likely to fit the data at hand. Using these points to seed the fitter optimizes the whole process and keeps the core of this analysis frequentist in nature.

The current limitation is that we do not have a global understanding of the likelihood surface. Exploring the surface with no knowledge outside local neighbors becomes a bottleneck if the surface is too complex. The solution is to skip the likelihood construction altogether. We need a Likelihood-free method, a neural network that can learn the underlying posterior distribution for a set of data points we provide. Since our aim for

this capstone is to provide a proof of concept, we constructed a similar toy problem to solve.

Problem Statement

Consider the Combined Gaussian

$$f(x) = \frac{N_1}{\sqrt{2\pi\sigma_1^2}} \exp(-\frac{(x-\mu_1)^2}{2\sigma_1^2}) + \frac{N_2}{\sqrt{2\pi\sigma_2^2}} \exp(-\frac{(x-\mu_2)^2}{2\sigma_2^2})$$

Our problem statement is as follows: given the Poisson fluctuated data derived from the above double Gaussian, can we develop and train Machine Learning Models that can take this data as input, and output the 6 parameter values $\vec{\theta} \rightarrow [N_i, \mu_i, \sigma_i](i = 1, 2)$ that went into constructing the underlying distribution.

Theory

2.2.1 Flows

There are a few different architectures that perform probabilistic generative modeling.

- Variational Auto-Encoders
- Generative Adversarial Networks
- Normalizing Flows

There are advantages to all approaches, but Normalizing Flows are most suited for the task at hand for the following reasons.

1. Normalizing flows give us access to the log-likelihood of a proposal $\vec{\theta}$, which we can directly use as its loss function. Minimizing the negative log-likelihood will tell us which $\vec{\theta_0}$ the model believes fits the data best. Such is not true for other probabilistic generative models; for such cases, we usually resort to minimizing a proposal distribution's Kullback-Leibler divergence (distance from proposal to "truth", which is usually just a Gaussian).
2. We can sample directly from the distribution and also have the ability to invert it to clearly see the mapping from base $\rightarrow$ proposal.

A normalizing flow works by learning a sequence of simple transformations that can transform a base distribution into a more complicated target distribution. To make this more concrete, consider a probability distribution $p(x)$, transforming $p(x) \rightarrow p(y)$ given a mapping $x \rightarrow y$ is *not* as trivial as replacing x with y. We also need to respect the probability conservation rule. The target distribution has to be distorted in such a

way that the local probability at any point remains the same. If $P_Y(y_0)$ is taller than $P_X(y_0)$, it has to be proportionally thinner, so probability is conserved.

$$P_X(x)dx = P_Y(y)dy \tag{1}$$

$$P_Y(y) = P_X(x)\left\|\frac{dx}{dy}\right\| \tag{2}$$

In higher dimensions,

$$P_X(x_1, x_2)dA_1 = P_Y(y_1, y_2)dA_2 \tag{3}$$

$$dA_2 = \|det[J_f(x)]\|\, dA_1 \tag{4}$$

$$P_Y(\vec{y}) = P_X(\vec{x})\, \|det[J_f(\vec{x})]\|^{-1} \tag{5}$$

$$\log P_Y(y) = \log P_X(x) + \log|det[J_f(\vec{x})]|^{-1} \tag{6}$$

Which we recognize as the change of variables formula. Next, consider the transformation U, which is a combination of successive transformations U_i. We can chain the formula all the way to the first transformation in the following manner.

$$U = U_1 \odot U_2 \odot ...U_{N-1} \odot U_N \tag{7}$$

$$\log U_N = \log U_{N-1} + \log|det[J_N(U_{N-1})]|^{-1} \tag{8}$$

$$\log U_{N-1} = \log U_{N-2} + \log|det[J_{N-1}(U_{N-2})]|^{-1} \tag{9}$$

$$\log U = \log U_1 + \sum \log|det[J_{i-1}(U_i)]|^{-1} \tag{10}$$

Equation (10) gives us our loss function. To calculate the log probability of a certain proposal point, we just need to calculate the log probability of the base distribution and sum over all the transformations' inverse Jacobian determinants. This introduces some constraints on what our transformations can look like. Any transformation we choose needs to meet the following criteria.

- It should be cheap to invert.
- It should be easy to compute its Jacobian Determinant.

For this capstone, we used the `nflows` package with many different transformations pre-computed. Let us take a look at one of the transformations: The `MaskedAffineAutoRegressiveTra`

First, it is useful to note that many of these transformations use affine coupling layers (i.e, simple affine scale + shift) as their derivatives are trivial to compute and store. The Masked Affine Autoregressive Transform is one such collection of affine mappings.

The core idea is simple: in a layer of nodes for the transformation, each node ONLY depends on previous inputs, meaning y_i only depends on $x_{<i}$. To make this concrete, let us look at what the Jacobian for such a transform looks like.

$$\begin{bmatrix} \frac{\partial y_1}{\partial x_0} & 0 & 0 & . & . & . \\ \frac{\partial y_1}{\partial x_1} & \frac{\partial y_2}{\partial x_1} & 0 & . & . & . \\ \frac{\partial y_1}{\partial x_2} & \frac{\partial y_2}{\partial x_2} & \frac{\partial y_2}{\partial x_2} & 0 & . & . \end{bmatrix}$$

The determinant for such a lower triangular matrix is just the product of the diagonal elements, which in log space becomes a sum. Inverting such a matrix is also trivial due to the affine nature of the mapping. $y_i(x_{<i}) = \sigma(x_{<i}) + \mu(x_{<i})x_{<i}$ where μ, σ themselves are some invertible `Multi-Layered Perceptron` (MLP).

The discussion above was for normalizing flows. We are interested in *posterior* probabilities $P(\vec{\theta}|\vec{d})$, so the entire discussion remains the same except now we *condition* an output on some observed data point.

2.2.2 Encoders

Let us take a look at the structure of an MLP 8. The input layer takes in a 6-dimensional input. The utility of these encoders exists within the hidden layer, where the MLP *learns* how to represent the input data with a smaller number of dimensions. In this example, a 6-D point gets compressed into a 3-D point. We call this the *latent representation* of the input data.

For example, imagine we trained an MLP to reconstruct data from input $\rightarrow$ output. It would take in an input, compress it into some latent space, and then decode the latent representation back into an output. If our objective training function is simply a Mean-Square error between the output and input, the MLP learns the most optimal weights and biases to compress and decompress this data. So in theory, you could encode some data, use it for other purposes, and then decode it later as required. This allows you to work in a lower-dimensional representation of the data, which you can manipulate more freely.

3 Method

There were 4 general steps in our workflow.

1. Generate training data
2. Encode the data into a latent space
3. Train a Conditional Normalizing Flow model on the latent space
4. Validate Results

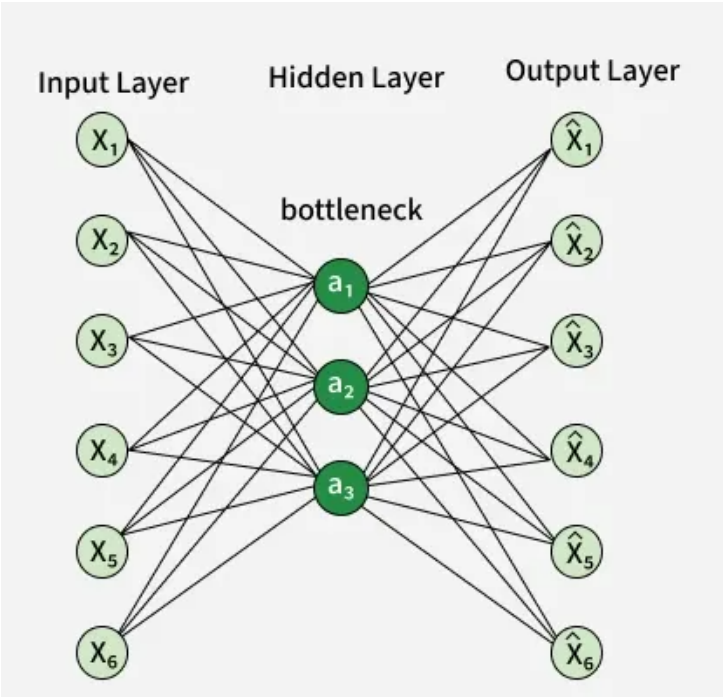


Figure 1: Architecture of an autoencoder (source: [1])

Generate Training Data

We followed the following procedure to generate the data.

- Define 6 uniform distributions for N_i, μ_i, σ_i (i=1,2) and populate M arrays with these 6 samples. The result is an Mx6 array with M unique samples. We will call this the $\vec{\theta}$ array.
- Use these arrays to generate M unique double Gaussian distributions, and save their bin-counts. We used 20 bins in our data.
- Poisson fluctuate each bin to simulate statistical uncertainty.
- Repeat the Poisson fluctuation process K times for **each** of the M unique experiments. We do this, as statistical uncertainty can deviate from the underlying data in many different ways; we want the CNF to see this, so it is able to differentiate noise from structure. We will call this the $\vec{d}$ array.
- Standardize both the $\vec{\theta}$ and $\vec{d}$ arrays using their respective z-scores.

It was important to use standardized data, as the relative scale differences between the parameters can cause problems during the encoding and flow stages. We also found that z-score standardization worked better than min-max scaling.

Auto-Encoder

- The MLP architecture is initialized with 5 layers and MSE Loss [input $\rightarrow$ middle $\rightarrow$ encoded $\rightarrow$ middle $\rightarrow$ Output].
- We encode $\vec{d}$ into a lower-dimensional latent space, and z-score standardize the latent space. This transforms $\vec{d} \rightarrow \vec{z}$.
- We save $\vec{z}$, $\vec{d_{mean}}$, and $\vec{d_{std}}$ so that later in the pipeline, when we run inference, we can use the same statistics to standardize the test data.

We will use $\vec{z}$ as the conditioning variable for the flow. Training and inference were run on the local Heimdall server, with the workload parallelized across 8 V100 GPUs.

Conditional Normalizing Flow

- The flow was initialized with 5 layers of [`MaskedAffineAutoregressiveTransform`, `ReversePermut`, `MaskedPiecewiseRationalQuadraticAutoregressiveTransform`]
- We start with a 6D Gaussian base distribution. We will transform this into the posterior.
- For training, we make the model learn $P(\vec{\theta}|\vec{z})$, using the objective (10) and then minimize it.
- Once training is finished, the model is saved in memory for future use.

We ran into some issues with losses exploding during training. So we normalized the gradients using `utils_grad_norm`. It is important not to clip gradients too harshly, or this slows down training significantly.

Validation

The validation procedure involved a few steps:

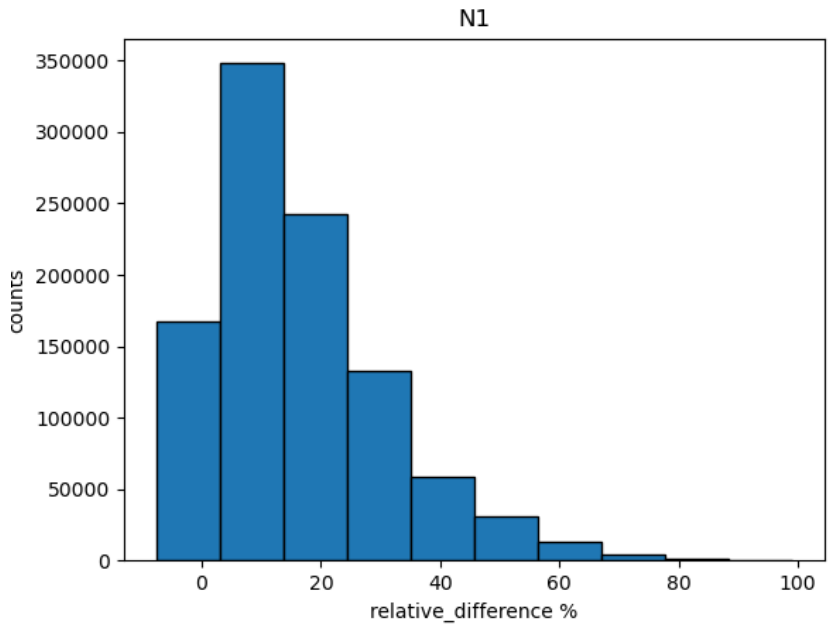
1. Generate Data with 1 truth value $\vec{\theta_0}$ and N corresponding double gaussian data.
2. Run data through AE + CNF. Get the theta posterior distribution for $\vec{\theta}$.
3. Use a Mode-Seeking algorithm to find a representative point $\vec{\theta_1}$ from the posterior. This point is what the model thinks best fits the data.
4. Find the percentage difference $\vec{p} = \frac{100*(\vec{\theta_1}-\vec{\theta_0})}{\vec{\theta_0}}$.
5. Plot $\vec{p}$ as a histogram

We do the above process many times and populate a histogram with the errors. Ideally, we should see a peak around 0 with a relatively small variance.

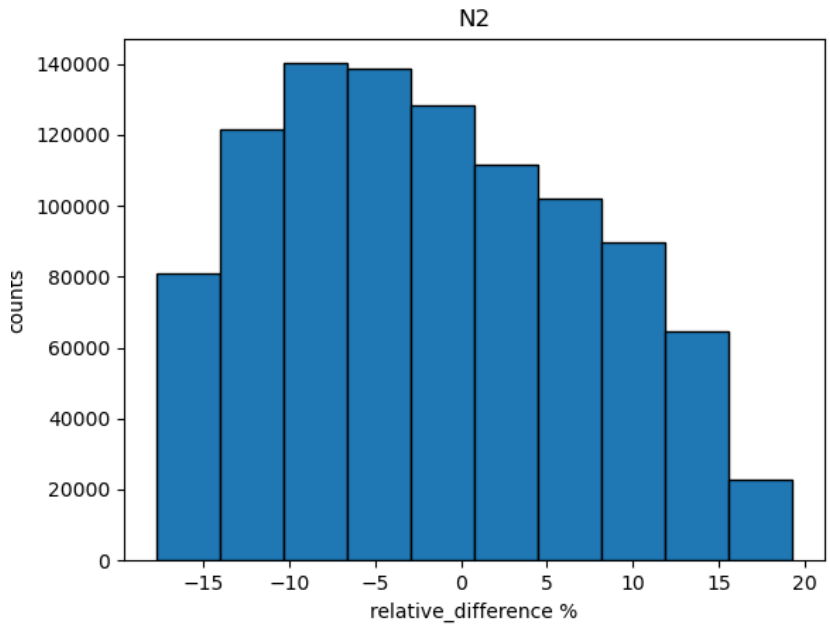
4 Results

Initial Results and Lessons

Our initial results were not ideal. Here are two plots that will help explain the problem.



(a) N_1



(b) N_2

Figure 2: Comparison of N_1 and N_2

There are 2 distinct problems with these plots.

1. The N2 Posterior is wide, this mean the model was UNABLE to learn anything for N2.

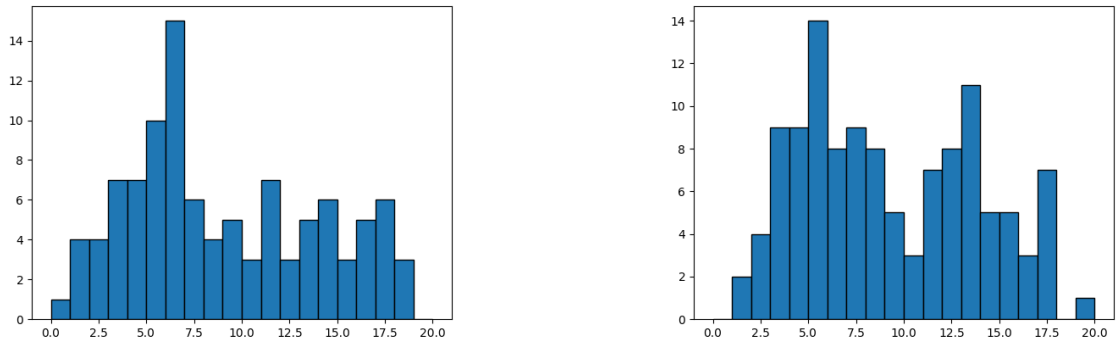
2. The mode values were not centered around 0.

In actuality, there were other problems, too. Sometimes the CNF would predict values more than $1e6\%$ off. This was fixed when we addressed the exploding gradient problem.

Importantly, these problems were not something that tweaking hyperparameters fixed. Something was fundamentally lacking in our solution.

Fixes and Final Results

Let us compare two plots produced by the same underlying double Gaussian, but statistically fluctuated.

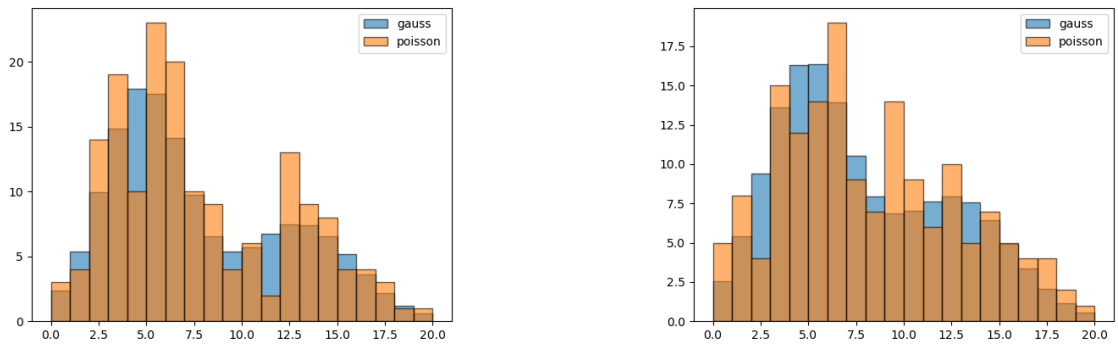


(a) Poor Separation

(b) Better Separation

Figure 3: Statistics Dominated Plots

Notice how, despite these 2 plots having the same underlying Gaussian. The separation between peaks is ill-defined due to statistical uncertainty. Without any prior knowledge, one may even mistake 3a as a singular noisy Gaussian. To illustrate my point, let us take a look at the reference Poisson distribution vs a Poisson distribution generated from $\vec{\theta_1}$.



(a) CNF Generated

(b) Real Poisson

Figure 4: Comparing Generated vs Truth

As seen in 4, when the separation between peak 1 and peak 2 is dominated by statistical uncertainty, the fault no longer lies with the CNF for failing to extract information from data; it lies with the data itself for not *containing* enough information.

The poor separation made it difficult to distinguish the N_2 peak, but it did not have as much of an effect on N_1, which makes sense. To fix this, we increased the available statistics by up-scaling the normalizations N_1, N_2.

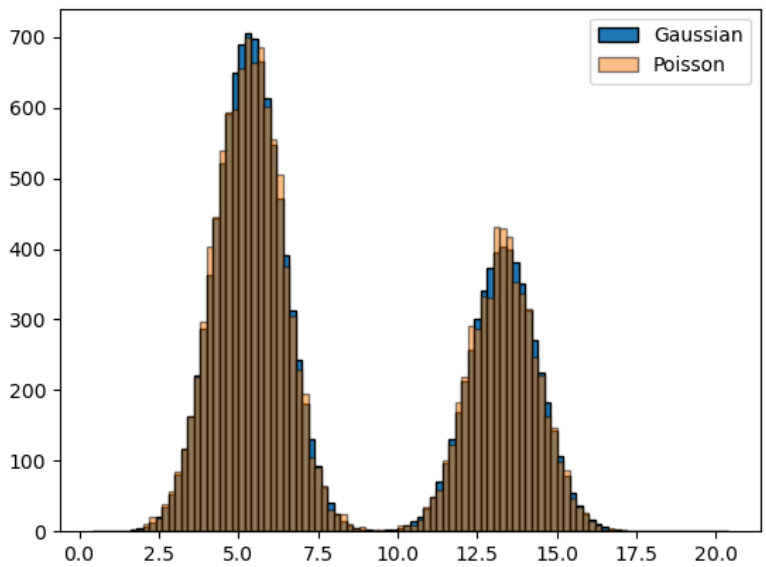
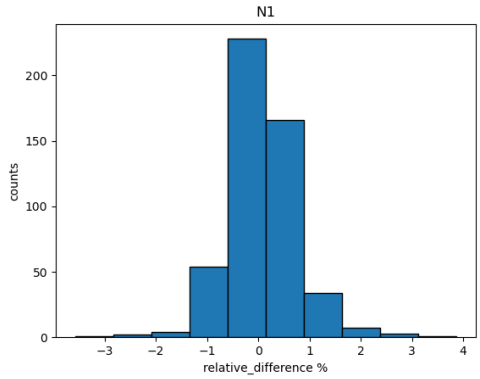


Figure 5: UpScaled Poisson

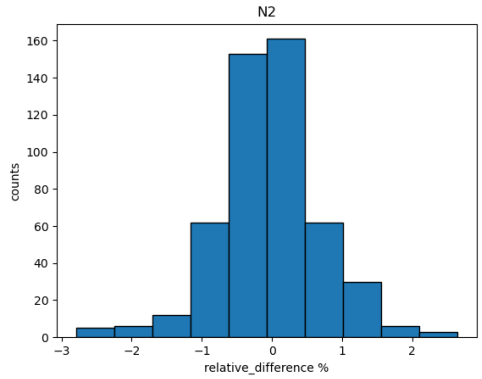
Simple statistical fluctuations will now not distort the underlying Gaussian to the point of unrecognizability. With this improvement, we repeated the validation procedure described above with one added step. Instead of directly computing the percentage

difference, we first pass in $\vec{\theta_1}$ to an `IMinuit` instance and fit it against $\vec{d}$. We take the best fit point and then compute its percentage difference with the ground truth value $\vec{\theta_0}$

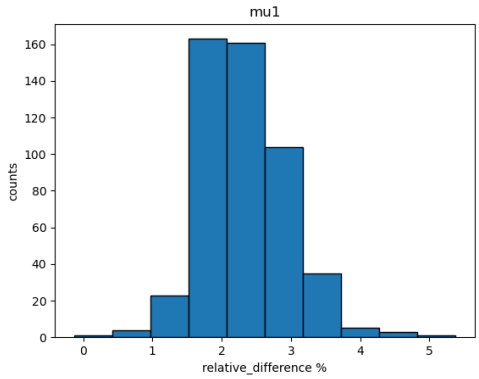
Here are the results.



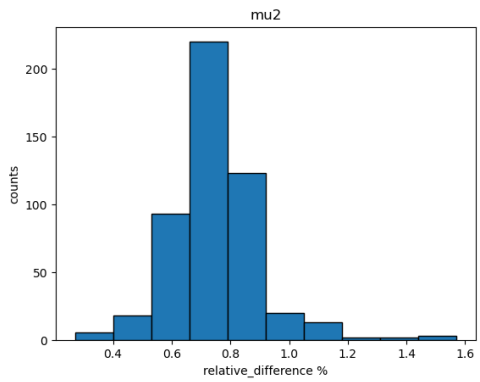
(a) N_1



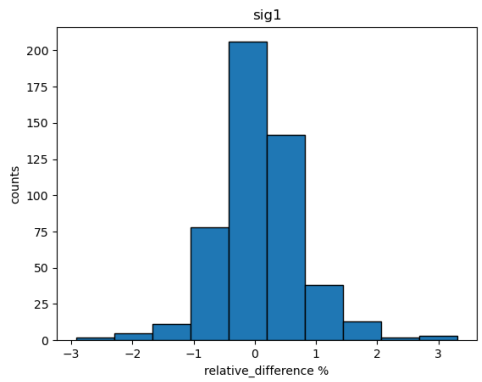
(b) N_2



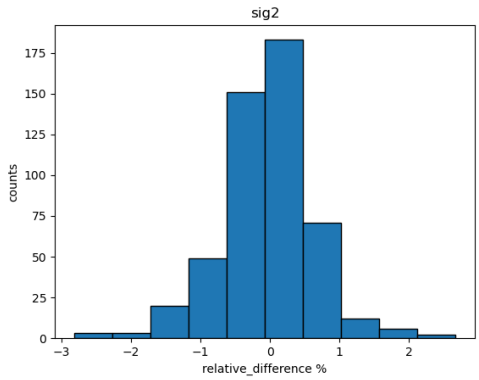
(c) μ_1



(d) μ_2



(e) σ_1



(f) σ_2

Figure 6: New Posterior correction plots

The problems we identified are all now resolved. There is still the issue that 6c is not maximized around 0, but that is more a fault with the fitter than the produced posteriors. It is important to note that, as a standard benchmark, we also tried fitting the generated data **without** the initial start points, and it never converged.

This problem was identified first by connecting it to a shallow χ^2 surface. During general Bayesian inference problems, one big bottleneck is not having enough statistics. If a likelihood surface is shallow due to insufficient statistics, we are unable to infer strong posteriors. There is simply not enough statistical "evidence" for a particular hypothesis.

This ties into an important lesson. We opted for a likelihood-free inference method so we can escape having to explicitly construct these expensive likelihood surfaces. But that does not automatically mean a machine learning model is able to accomplish what a traditional inference procedure cannot (given infinite resources). Even though we do not construct an explicit surface, it can and *will* become a bottleneck to ML analysis as it would for a more traditional method, given the circumstances.

5 Extension

Strictly speaking, my capstone is effectively complete. Our original intentions were to simply provide a proof of principle. A double Gaussian is simple enough to visualize, but sufficiently nontrivial to challenge standard fitting procedures. Regardless, the next natural step is to apply this to the original problem, as was its intended purpose. We will now try to enhance the global sterile neutrino fitter with this machinery.

The general steps are the same as before

- Generate Data
- Train the AE + CNF combination
- Produce inference results
- Seed a fitter with the inference results and compare its new performance with its standard performance

There are, however, a few problems we need to address first.

1. The pseudo-experiment generation and fitting functionality requires a large number of custom package dependencies.
2. The *entirety* of this framework exists on top of `novasoft`; a complex remote ecosystem we cannot containerize and run on `Heimdall` where the ML architecture resides.
3. We also cannot move the ML models to the `novagpvms` as their Linux containers have poor PyTorch support.

So, the solution we opted for is to do the transfer manually. We will generate the data on the novagpvms and transfer a `ROOT` file to Heimdall, run training + inference, and then port the results back onto the novagpvms where we will run the final fit.

Data Generation and Training

The data generation is identical to the previous section. The only difference is that we also add systematic uncertainties. Since this data is a lot more expensive to generate, we compromised by reducing the number of repeated experiments per unique sample. The theta values we are fit for are

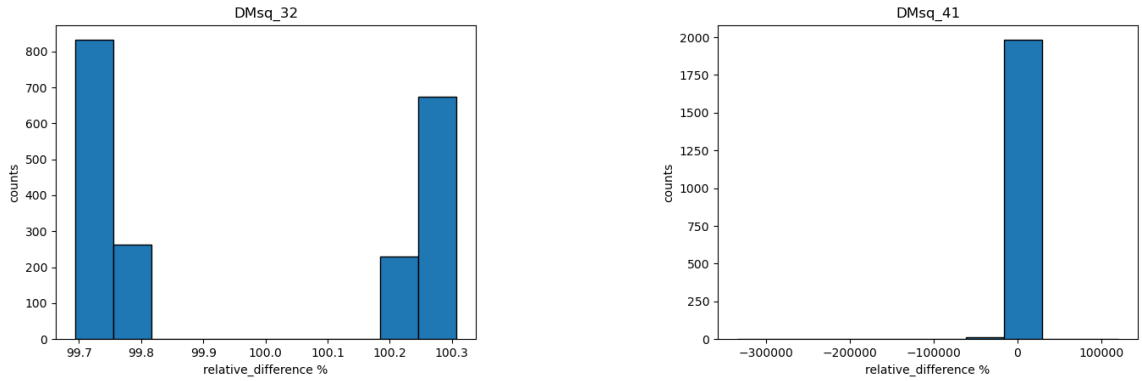
$(\Delta m^2_{41}[-3,2], sin^2(\theta_{24})[-2,0], sin^2(\theta_{34})[-5,0]$, $\delta_{24}[0,2\pi], sin^2(\theta_{23})[0,1], \Delta m^2_{32}[2.3e-3, 2.8e-3])$

Where we generate all the data within a uniform prior, but Δm^2_{41}, $sin^2(\theta_{24})$, $sin^2(\theta_{34})$ are in log space as they can take on much smaller values to allow for sterile oscillations.

We get $\vec{d}$ by concatenating the spectra of 9 different channels. We have 4 sets of data from the near detector and 4 from the far detector, and a muon neutrino channel only from the near detector.

Initial Results and Fixes

Running the same procedure as before, but with no fitting, just plain posterior errors. Let us take a look at 2 of the $\vec{p}$ plots.



(a) DM^2_{32} (b) DM^2_{41}

Figure 7: Statistics Dominated Plots

Clearly, something went wrong here. We are predicting garbage; let us try to understand where the bottlenecks are.

It will be illuminating to plot $\vec{d}$.

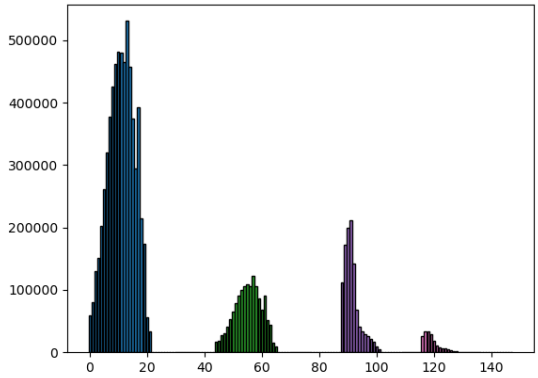


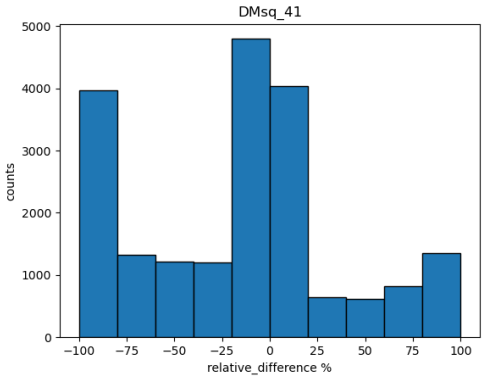
Figure 8: Encoder

We see the first problem: the 9 channels are bound tightly together in 4 discontinuous clusters. The CNF will have a hard time parsing through such context. The natural question may now be, "Wasn't the Autoencoder designed to reduce dimensionality and fix these issues?" The answer is no, the autoencoder is specialized in reducing dimensionality *purely* for the cause of reconstruction. It does not know anything about the CNF and could easily clutter up the latent space with information relevant to reconstruction but not to the CNF. The solution is to change the encoder's objective function from MSE Loss to log likelihood loss from the CNF. What we need to do is train both the encoder and the flow together.

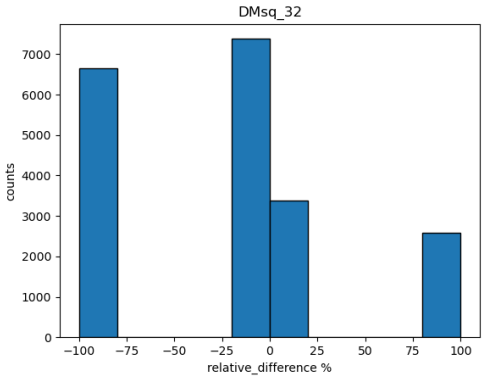
The next problem is the way we are mode seeking. The current implementation is naive; we simply marginalize the posteriors into 6 independent distributions and then find each of their modes. This worked fine for our toy example, as the distribution was unimodal. Such is not the case here; the data is degenerate and heavily influenced by systematics. We need a better mode-seeking method. The solution is a combination of `DBSCAN` and log-likelihood ranking. The core idea is that we generate samples $\vec{\theta_1}$ like before, but now, we cluster them using DBSCAN and then pass them through the CNF again. Equation 10 will tell us how likely a sample is given the data. We then pick out the top K points from each cluster to pass to the fitter.

Final results

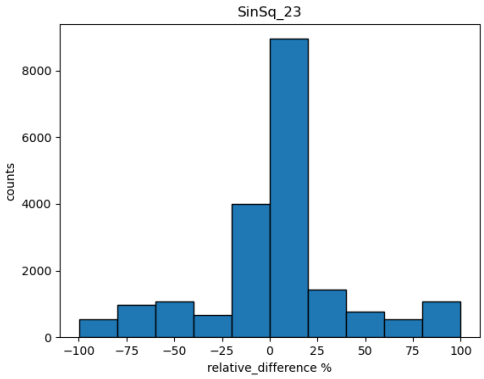
After making the above changes, and some extra hyperparameter tweaks. We get the following percentage error plots.



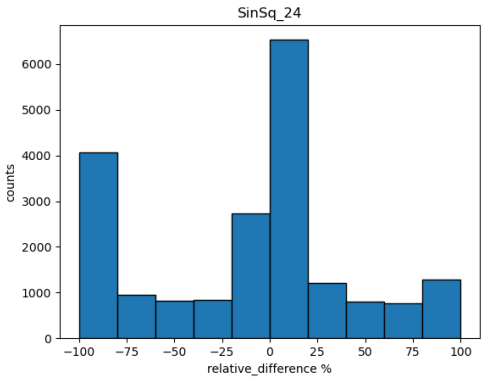
(a) DM^2_{41}



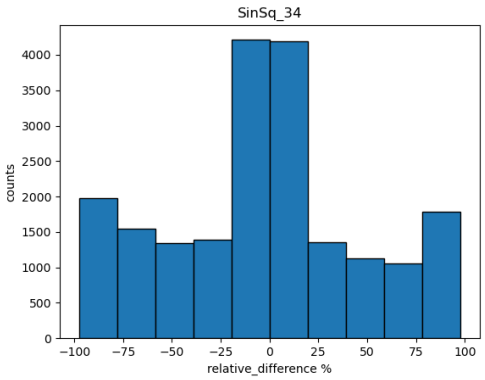
(b) DM^2_{32}



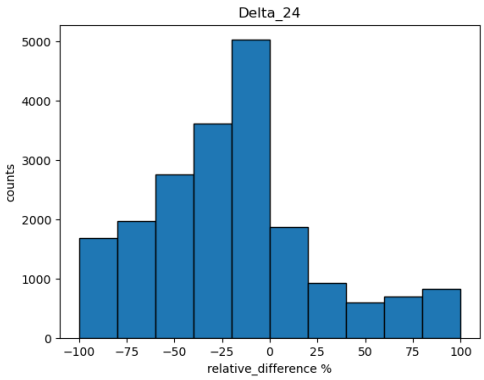
(c) $sin^2(\theta_{23})$



(d) $sin^2(\theta_{24})$



(e) $sin^2(\theta_{34})$



(f) δ_{24}

Figure 9: New Posterior correction plots

That is a lot better. It now looks similar to our roadblock with the toy problem. The posteriors indicate a lack of statistics. There is, however, a silver lining; none of the plots are flat. They all have a mode near 0, except Δm^2_{32}, but that is to be expected, given the degeneracy between mass orderings. It is reassuring to know that the model is not hallucinating results.

Future Plans

As of now, this is as far as we have gone. We are yet to test this on the global fitter instance. That will be the next plan of action.

6 Appendix

Here are the relevant git repositories for this analysis.

CNF + AE

Prediction and Fitting Machinery → Visit branch feature/karthiva/Prediction_Gen

Final Architecture of Models

Hyperparameter	**Value**
Batch Size	512
Gradient Clipping	True
Epochs	30
CNF Layers	8
Layer Types	ReversePermutation MaskedAffineAutoregressiveTransform MaskedPiecewiseRationalQuadraticAutoregressiveTransform
Hidden Width	30
Spline Num Bins	24
Spline Tails	Linear
Spline Tail Bound	3.5
AE Middle Dim	111
AE Output Dim	74
Optimizer	Adam
Learning Rate	0.001
Betas	(0.9, 0.999)
Epsilon	10^{-8}
Weight Decay	0
AMSGrad	False

Table 1: Model and Training Hyperparameters

References

[1] GeeksforGeeks. Autoencoders in machine learning, 2025. Accessed: 2026-04-22.